

一、基础知识

1. Internet

1.1 IP

internet protocol

IP 是通过 Internet 发送的所有数据（数据包）的基础通信系统

1.2 URI、URL、URN

URI: Uniform Resource Identifier, 统一资源标识符, 允许资源处于互联网的任何位置

URL: Uniform Resource Locator, 统一资源定位符, 指出资源的一个拷贝的位置

URN: Uniform Resource Name, 是资源的唯一名称, URN 需要一个支持的基础设施来解决资源位置问题, 这就是为什么它还没有被广泛采用

URI 是 URL 和 URN 的超集

1.3 DNS

DNS 是唯一同时使用 TCP 和 UDP 的

- 在 DNS 域中的主备冗余机制中, 进行同步通信时使用 TCP 协议, 原因在于其可靠性
- 在用户通过浏览器访问 DNS 服务器请求 IP 地址时使用 UDP 协议

1.4 HTTP 1.1、HTTP /2、HTTP /3 协议

HTTP 1.1 支持持久连接 (HTTP 1.1 的默认模式使用带流水线的持久连接), 在一个 TCP 连接上可以传送多个 HTTP 请求和响应, 减少了建立和关闭连接的消耗和延迟。HTTP 1.1 允许在持久性连接上进行可选的请求管道传递, 在响应到达之前, 可以将多个请求放入队列。

HTTP /2: (原名 HTTP /2.0) 是万维网使用的 HTTP 网络协议的主要修订版

HTTP /2 和 HTTP 1.1 的不同:

- HTTP /2 不需要对现有的 web 应用程序的工作方式进行任何更改, 但是新的应用程序可以利用新特性来提高速度
- HTTP /2 保留了 HTTP 1.1 的大部分高级语法, 例如方法、状态码、头字段和 URI。The element that is modified is how the data is framed and transported between the client and the server.
- HTTP /2 可以最小化渲染整个页面所需的请求数, 使网站更加高效
- HTTP /2 的第一稿 (是 SPDY 的副本) 中的其他性能改进来自于请求和响应的多路复用, 以避免 HTTP 1 中的首行阻塞问题 (即使在使用 HTTP 管道时)、首部压缩和请求的优先级排序

HTTP /3:

- HTTP over QUIC (Quick UDP Internet Connection, 是谷歌制定的一种基于 UDP 的低时延的互联网传输层协议)
- 基于 UDP

2. 网络编程相关

2.1 网络机器人

又称网页爬虫（Web crawler）、网页蜘蛛。

定义：是一种通过有条理、自动的方式浏览网络的计算机程序

功能：

- 在网络上收集页面
- 支持搜索引擎，进行数据挖掘

对象：

- 文本、图片、视频等
- 链接结构

爬虫的结构：

- URL 队列（URL Frontier）：containing URLs yet to be fetches in the current crawl。首先，将种子集存储在 URL 队列中，然后爬虫从种子集中获取 URL
- DNS：域名服务解析，查找域名的 IP 地址
- 爬取：一般来说是用 http 协议来爬取 URL
- 解析：页面被解析，文本、图片、视频等和链接被提取

网络爬虫排除标准：

Robots Exclusion Protocol，网站通过 Robots 协议告诉搜索引擎哪些页面可以抓取，哪些页面不能抓取。

Robots 协议是国际互联网界通行的道德规范，基于以下原则建立：

- 搜索技术应服务于人类，同时尊重信息提供者的意愿，并维护其隐私权；
- 网站有义务保护其使用者的个人信息和隐私不被侵犯

robots 是一个协议。robots.txt 文件是一个文本文件，放置在网站根目录下

3. Web

3.1 Web 的发展

web1.0: Web of documents，网络资源从信息产生者到信息客户的单向流程

web2.0: Web of persons，所有人既是网络资源的信息产生者又是信息客户

web3.0: Web of data（semantics），语义网，智能语义程序介入网络资源流程，更有针对性地发送信息和获得信息

Web 2.0 特点：Web 2.0 是一个架构在知识上的环境，人与人之间交互而产生出的内容，经由在服务导向的架构中的程序，在这个环境中被发布、管理和使用。

典型应用：Facebook、wiki 百科、豆瓣、土豆网、微博

局限性：

- 过度饱和：数据太多，有大量无用信息和垃圾信息

- 概念偏差：审查宽松导致“标题党”“炒冷饭”现象
- 时间局限：数据信息太多导致浪费时间或时间不充裕
- 互动模式：对人际关系有所影响
- 过度开放：个人隐私被滥用

(18 年) **Web 3.0 特征**:

- 语义化：对网络内容有机理理解上语义化的定义
- 3D
- 人工智能：构建于以后需要算法进行相应处理，开发易用
- 去中心化：源于安全问题的考虑

3.2 MEAN

MEAN 是一个 Javascript 平台的现代 Web 开发框架总称

- M: MongoDB, 是一个使用 JSON 风格存储的数据库, 非常适合 javascript (JSON 是 JS 数据格式)
- E: ExpressJS, 是一个 Web 应用框架, 提供有帮助的组件和模块帮助建立一个网站应用
- A: AngularJS, 是一个前端 MVC 框架
- N: Node.js, 是一个并发、异步、事件驱动的 Javascript 服务器后端开发平台

二、浏览器端

1. HTML/XHTML

1.1 结构、表现、行为

HTML 结构: 一般分为三部分, 用 html 标签包裹

- DOCTYPE: 声明了使用的文档类型定义 (DTD)
- Head: 包含元数据, 一般只需要 title 标签
- Body: 需要渲染的文本

HTML 工作流程:

1. HTML 通过标记符标记要显示的网页的各个部分, 通过添加标记符来确定内容格式, 从而令浏览器知道如何显示网页
2. 浏览器按顺序阅读 HTML 文件, 然后根据 HTML 标记符解释和显示各种内容, 这就是语法分析过程
3. HTML 中的超链接功能使得网页之间能够链接起来从而提供跳转功能

1.2 基本语法、常用标记

常用标记:

1. 注释

```
1 | <!-- 注释 -->
```

上线产品不要包含注释:

- 注释（明文传播）会提高页面被解读的风险
- 注释会增加传输带宽消耗

2. title

```
1 | <title></title>
```

- 描述网页标题
- 位于 head 标签之间
- 在浏览器的 title bar 上显示
- 精简且语义化强的描述能吸引爬虫

3. meta

```
1 | <meta>
```

- 描述网页元数据
- 位于 head 标签之间
- 其 Charset 属性在实际中非常重要

4. p

```
1 | <p></p>
```

- 用于表示段落，位于 body 之间

5. br

```
1 | <br>
```

用于强制显示换行，一般用于块级元素之间，要马上以 /> 结尾

6. a

```
1 | <a href=""></a>
```

- 指向其他页面的链接
- 使用 href 属性来指定目的 url，可以是绝对路径或相对路径
- 行级元素，要在 p 或 h1 等块级元素中
- 原则要让链接描述更具表属性从而更好理解

7. img

```
1 | <img src="" alt="">
```

- 向页面中插入图片
- src 属性指定了图片 url
- xhtml 还需要 alt 属性
- 在 a 标签之间放置会变为链接
- title 属性指定了可选 Tooltip

- .git 和 .png 无损, .jpeg 和 .webp 有损

8. table, tr, td, th, caption

- table 代表表体
- caption 是标题
- tr 表示一行, th 表示表头, td 表示值

9. dl, dt, dd

- dl 代表一系列术语的定义
- dt 表示每一个术语, dd 中放定义

10. quotation

- blockquote 是段落引用, 可以用于 p 标签外将段落变为引用格式 (斜体)
- q 是行内引用, 可以用于单词两边, 效果是添加上双引号

11. form

```
1 | <form action="" method="" target=""></form>
```

- 用于在网页上创建 GUIs, 目的通常是向用户请求信息
- 其参数指定 js 如何将信息传至服务器
 - action 指定 submit 按下后数据送往的地点
 - method 默认为 GET, 能以 {form_data} 形式传出, data 全为 ASCII 码且不超过 100 字符; POST 将 data 放置于 body 部分
 - target 指定响应页面的打开方式 (当前或新窗口)
- form 元素有 button, checkbox, text, radio

12. input

```
1 | <input type="text" hidden>
```

- 强大, 但从语义化角度讲不是好的设计
- 具体样式渲染要等读到 type 属性值才能确定
- hidden 类型在 cookies 被禁用时有优势, 能通过脚本语言设置其值返回给服务器, 同时不被用户看到
- 不需要 JS, 新增原生日期、颜色选择器以及其他新类型, 提供客户端验证机制和旋转框、滑块

13. canvas

```
1 | <canvas></canvas>
```

- 提供本地绘画功能
- 可用于图表、图像、动画和像素处理等
- 支持 2D 和 3D 效果

14. video&audio

- 提供非插件式视频&音频支持, 可以用 JS 进行操作, CSS 进行渲染
- 能像添加图片一样添加
- 支持不同编解码器, 但有多源元素和回退

- 可以使用内容，有完整的 JS API 供使用

(14 年) 为什么不用 table 布局：

1. table 比其它 html 标记占更多的字节。
2. table 会阻挡浏览器渲染引擎的渲染顺序（会延迟页面的生成速度，让用户等待更久的时间）
3. table 里显示图片时需要你把单个、有逻辑性的图片切成多个图。
4. 在某些浏览器中，table 里的文字的拷贝会出现问题。
5. table 会影响其内部的某些布局属性的生效（限制页面设计的自由性）
6. 一旦学了 CSS 的知识，你会发现使用 table 做页面布局会变得更麻烦。
7. “table 对”对于页面布局来说，从语义上看是不正确的（它描述的是表现，而不是内容）
8. table 代码难以阅读。
9. table 一旦设计完成就变成死的，很难通过 CSS 让它展现新的面貌。

为什么不直接用引号而用 <q>：

1. XHTML 不应该包含字面引号字符，它们应该被写成 "
2. 用 <q> 能让我们将 CSS 样式应用于引号

(18 年) 表单隐藏域的作用：

1. 所有输入字段都会发送回服务器，包括隐藏字段
2. 这是一种包含用户不需要查看（或者你不希望他看到）的信息的方法
3. 在提交表单之前，可以通过编程方式（JavaScript）设置隐藏字段的值

隐藏域在页面中对于用户是不可见的，在表单中插入隐藏域的目的在于收集或发送信息，以利于被处理表单的程序所使用。浏览者单击发送按钮发送表单的时候，隐藏域的信息也被一起发送到服务器。

1.3 html 语义化

如何理解：

- 简单来说就是用特定的标签体现相对应的功能
- 对人而言，增强可读性，对开发者更友好，在没有 CSS 的情况下也能较好地呈现网页的内容结构与代码结构，有利于团队开发和维护
- 对机器而言，有利于 SEO，能让搜索引擎更好的获取更多有效信息，支持读屏软件，方便其他设备解析，有利于无障碍阅读，提高可访问性

原因：

1. 有利于 SEO
2. 开发维护体验好
3. 用户体验更好
4. 更好的可访问性，方便任何设备对代码进行解析

1.4 html5 新特性

why html5：

1. 向后兼容，即支持 h5 的浏览器也可以运行更低的版本，因为 h5 采用务实的方法，解决常见的现实问题

- 2. 更加简化
- 3. 是通用的，支持所有语言
- 4. 有更少的插件
- 5. 安全

(14 年) 改进 (新功能) **:

- 新元素
 - article、nav、header、video、svg 等
- 新属性
- 完全支持 CSS3
- video 和 audio
- 2D/3D 制图
- 本地存储
- 本地 SQL 数据
- Web 应用

HTML5 应用程序缓存优势:

- 离线浏览，用户可在应用离线时使用它们
- 速度，已缓存资源加载得更快
- 减少服务器负载，浏览器将只从服务器下载更新过或更改过的资源

2. CSS

2.1 why (14 年)

- 更加灵活、精简和清晰
- 格式上是基于 formatting tool 的布局
- 方便管理多个文档
- 使用类选择器来节省时间
- 更多格式化的机会

CSS 工作流程:

1. 浏览器先加载 html 并解析生成 DOM
2. 浏览器加载 CSS 并解析生成 CSSOM，处理时需要遍历处理 (继承)
3. 将 DOM 和 CSSOM 整合后 (为 DOM 附加样式) 呈现在浏览器中

2.2 css2.1, css3 新特性

css2.1 新特性:

- 元素的绝对定位、相对定位和固定定位
- media types 的概念
- 新的字体属性，比如阴影 (shadows)

(18 年) **css3 新特性:**

- 模块化，便于浏览器浏览
- 增加大约 50 个模块（准备情况各不相同）
- 使用特定于浏览器的前缀，直到最终确定
- 显著提高性能

举例：

- Border radius (rounded corners) without images
- Gradients, 渐变
- Multi-column layout, 多栏布局
- Transformations and transitions, 变换和过渡
- Web Fonts
- Media Queries, 媒体查询

2.3 CSS 盒模型

网页中的所有项目都会生成不可见的“盒”，必须弄清楚所有这些盒如何适合你的页面，就像拼图一样。

1. 浮动定位
2. 绝对定位
3. 相对定位

总元素的宽度 = 宽度 (content) + 左填充 (padding-left) + 右填充 (padding-right) + 左边框 (border-left) + 右边框 (border-right) + 左边距 (margin-left) + 右边距 (margin-right)

总元素的高度 = 高度 (content) + 顶部填充 (padding-top) + 底部填充 (padding-bottom) + 上边框 (border-top) + 下边框 (border-bottom) + 上边距 (margin-top) + 下边距 (margin-bottom)

2.4 响应式网页设计，主要手段

响应式网页设计 (RWD) 是一种网页设计方法，旨在在各种设备（移动电话到台式计算机）上提供最佳的观看体验，即易于阅读和导航，并且调整大小，平移和滚动的操作最少

围绕着三个概念建立：

1. 流畅或灵活的页面布局，根据浏览器窗口大小成比例缩放。
2. 灵活、比例适中的图像和视听媒体。
3. 使用 CSS3 媒体查询 (media query)，确定浏览器屏幕的宽度并作出相应的调整。

策略：

- 一切弹性化
- 流体网格
- 液态图片技术
- css3 媒体查询
- 响应式图片

2.5 优先级顺序和继承关系

优先级：浏览器缺省 < 外部样式表 < 内部样式表 < 内联样式

样式表：类选择器 < 类派生选择器 < ID 选择器 < ID 派生选择器

有 !important 的声明优先级高于一切

继承：特定的 CSS 属性向下传递到子孙元素（文字属性、列表相关属性、颜色）

百分比不能继承

2.6 布局

流式布局

即百分比布局，元素宽度并不是固定的，而是按照分辨率宽度进行调整，但是整体布局并没有发生改变

响应式布局

曾经流行，不再是必不可少，原因：

- 公司研发人员越来越充足，可以在 PC 端和移动端实现两套布局，分项目进行维护。
- 响应式布局在适配上越来越简单。

仍有存在的价值：

- 移动端碎片化的现象将会无限期存在
- 前端也必然进入物联网领域，任何设备界面的响应布局都将会成为关键挑战。
- 响应式布局是 CSS 逐步发展中的一环，体现了 CSS 的灵活性（媒体查询）

优势：

- 网站可用性得到提升，同时与移动优先设计以及内容策略能够非常好的融合在一起。
- 简化服务器端
- 更容易维护
- 只提供一个入口给搜索引擎
- 能够支持未知设备

缺点：

- 性能：
 - 兼容各种设备工作量大，效率低下
 - 代码累赘，会出现隐藏无用的元素，加载时间加长
- 限制应用的复杂性
 - 折衷性质的设计解决方案，多方面因素影响而达不到最佳效果
- 一定程度上改变了网站原有的布局结构，会出现用户混淆的情况

3. JavaScript *

3.1 基本语法

避免全局变量

先声明，或声明即初始化

不要声明 Number、String、Boolean

不要用 new Object()

注意自动类型转换

用 === 比较

默认参数

用 defaults 结束 switch

不要用 eval()

方法字面量 (method literals)

```
1  var baz = {  
2    f: function () {  
3      return arguments.callee;  
4    }  
5  };  
6  
7  baz.f();  
8  
9  //输出:  
10 f () {  
11   return arguments.callee;  
12 }
```

优点:

- 可以通过对象字面量将相关联的函数组合起来，让代码更加结构化并且易读
- 如果模块过于臃肿，可以使用方法字面量重写代码

示例:

```
1  var lightbulbAPI = {  
2    toggle: function() {  
3      console.log("toggle");  
4    },  
5    getState: function() {  
6      console.log("getState");  
7    },  
8    off: function() {  
9      console.log("off");  
10   },  
11   on: function() {  
12     console.log("on");  
13   },  
14 }
```

```
14     blink: function() {
15         console.log("blink");
16     }
17 };
18 lightbulbAPI.toggle();
```

控制台输出: toggle

提升 (Hoisting)

引擎会在解释 JavaScript 代码之前首先对齐进行编译, 编译过程中的一部分工作就是找到所有的声明, 并用合适的作用域将它们关联起来, 这也正是词法作用域的核心内容。

提升是 JavaScript 将声明移至顶部的默认行为

函数作用域的提升:

```
1  var x = 1;
2  (function () {
3      console.log(x);
4      var x = 2;
5  })();
6  //undefined
7
8  //相当于
9  var x = 1;
10 (function () {
11     var x;    //声明被提升, 此时x是undefined.
12     console.log(x);
13     x = 2;    //初始化还在这里
14 })();
```

只会提升函数声明, 而不会提升函数表达式。

this

- 在函数体中, 非显式或隐式地简单调用函数时, 在严格模式下, 函数内的this会被绑定到 undefined 上, 在非严格模式下则会被绑定到全局对象 window/global 上

```
1  function f1() {
2      console.log(this);
3  }
4  function f2() {
5      "use strict";
6      console.log(this);
7  }
8  f1(); // window
9  f2(); // undefined
```

- 一般使用 new 方法调用构造函数时, 构造函数内的 this 会被绑定到新创建的对象上

```

1 function Foo() {
2   this.bar = "Lucas"
3 }
4 const instance = new Foo()
5 console.log(instance.bar)
6 //Lucas

```

- 一般通过 call/apply/bind 方法显示调用函数时，函数体内的 this 会被绑定到指定参数的对象上

```

1 const foo = {
2   name: 'lucas',
3   logName: function() {
4     console.log(this.name)
5   }
6 }
7 const bar = {
8   name: 'mike'
9 }
10 console.log(foo.logName.call(bar))
11 //mike

```

- 一般通过上下文对象调用函数时，函数体内的 this 会被绑定到该对象上

```

1 var student = {
2   name: 'Lucas',
3   fn: function() {
4     return this
5   }
6 }
7 console.log(student.fn() === student);
8 //true
9
10 var person = {
11   name: 'Lucas',
12   brother: {
13     name: 'Mike',
14     fn: function() {
15       return this.name
16     }
17   }
18 }
19 console.log(person.brother.fn());
20 //Mike
21
22 var o1 = {
23   text: 'o1',
24   fn: function() {
25     return this.text
26   }
27 }
28 var o2 = {
29   text: 'o2',
30   fn: function() {
31     return o1.fn()
32   }
33 }

```

```

33 }
34 var o3 = {
35   text: 'o3',
36   fn: function() {
37     var fn = o1.fn
38     return fn()
39   }
40 }
41 console.log(o1.fn()) //o1
42 console.log(o2.fn()) //o1
43 console.log(o3.fn()) //undefined

```

- 在箭头函数中, this 的指向是由外层 (函数或全局) 作用域来决定的

```

1  var foo = {
2    fn: function(){
3      setTimeout(() => {
4        console.log(this) //{fn: f}
5      })
6    }
7  }
8  console.log(foo.fn()) //undefined
9
10 //对比
11 const foo = {
12   fn: function(){
13     setTimeout(function() {
14       console.log(this) //window
15     })
16   }
17 }
18 console.log(foo.fn()) //undefined

```

- this 指向最后调用它的对象

```

1  var foo = {
2    bar: 10,
3    fn: function() {
4      console.log(this) //windows
5      console.log(this.bar) //undefined
6    }
7  }
8  var fn1 = foo.fn;
9  fn1();
10
11
12 var foo = {
13   bar: 10,
14   fn: function() {
15     console.log(this) //{bar: 10, fn: f}
16     console.log(this.bar) //10
17   }
18 }
19 foo.fn();

```

this 优先级

- 显式绑定：通过 call、apply、bind、new
- 隐式绑定：根据调用关系确定 this 指向

```
1 function foo(a) {  
2   console.log(this.a)  
3 }  
4 var obj1 = {  
5   a: 1,  
6   foo: foo  
7 }  
8 var obj2 = {  
9   a: 2,  
10  foo: foo  
11 }  
12 obj1.foo.call(obj2) //2  
13 obj2.foo.call(obj1) //1
```

```
1 function foo(a) {  
2   this.a = a  
3 }  
4 var obj1 = {}  
5 var bar = foo.bind(obj1)  
6 bar(2)  
7 console.log(obj1.a) //2
```

```
1 function foo() {  
2   return a => {  
3     console.log(this.a)  
4   };  
5 }  
6 var obj1 = {  
7   a: 2  
8 }  
9 var obj2 = {  
10  a: 3  
11 }  
12 var bar = foo.call(obj1)  
13 console.log(bar.call(obj2)) //2
```

```

1  var a = 123
2  const foo = () => a => {
3    console.log(this.a)
4  }
5  const obj1 = {
6    a: 2
7  }
8  const obj2 = {
9    a: 3
10 }
11 var bar = foo.call(obj1)
12 console.log(bar.call(obj2)) //123 defined

```

3.2 严格模式

ES6 模块自动采用，其他的可以用"use strict" 指令，"use strict" 的目的是指定代码在严格条件下执行。

(18 年) **严格模式有以下限制：**

- 变量必须声明后再使用。
- 函数的参数不能有同名属性，否则报错
- 不能使用 with 语句
- 不能对只读属性赋值，否则报错
- 不能使用前缀 0 表示八进制数，否则报错
- 不能删除不可删除的属性，否则报错
- 不能使用 delete prop 删除变量，会报错，只能删除属性 delete global[prop]
- eval 不会在它的外层作用域引入变量
- eval 和 arguments 不能被重新赋值
- arguments 不会自动反映函数参数的变化
- 不能使用 arguments.callee 和 arguments.caller
- 禁止 this 指向全局对象
- 不能使用 fn.caller 和 fn.arguments 获取函数调用的堆栈
- 增加了保留字（比如 protected，static 和 interface）

为什么要使用严格模式？

- 消除代码运行的一些不安全之处，保证代码运行的安全；
- 提高编译器效率，增加运行速度；
- 为未来新版本的 Javascript 做好铺垫。

3.3 "first-class" functions

头等函数

JS 里面的 function 是基础的变量类型之一，可以作为参数或返回值，可以赋给变量，可以做闭包

"first-class" functions 以 functions 来构造对象

function 对象是 JS 里面的固有对象，所有的函数实际上都是一个 function 对象

3.4 事件驱动编程

JS 是基于对象 (object-based) 的语言, 这与 Java 不同, Java 是面向对象的语言。而基于对象的基本特征, 就是采用事件驱动 (event-driven)。它是在图形界面的环境下, 使得一切输入变得简单化。通常鼠标或热键的动作我们称之为事件 (event), 而由鼠标或热键引发的一连串程序的动作, 称之为事件驱动 (event driver)。对事件进行处理的程序和函数, 我们称之为事件处理程序 (event handler)

在事件驱动的应用程序中, 通常有一个主循环侦听事件, 然后在检测到其中一个事件时触发回调函数。

事件驱动程序可以用任何编程语言编写, 尽管在提供高级抽象 (如闭包) 的语言中, 该任务更容易。

一种程序执行序列由事件决定的编程范式

(14 年) **event handler (事件处理程序) 注册的方法:**

- inline:

```
1 | <a href="somewhere.html" onClick="myFunction()">
```

- traditional:

```
1 | element.onclick = myFunction;
```

- DOM 2:

```
1 | element.addEventListener("click", myFunction);
```

- IE: (evil enough!)

```
1 | element.attachEvent("onclick", myFunction);
```

- JQuery, Prototype...and so on: (preferred)

```
1 | jQuery.on();  
2 | Event.observe("target", "click", myFunction);
```

3.5 面向对象

JS is more object-oriented: noun.verb(), less procedural: verb(noun)

3.6 匿名函数

"Anonymous" functions (expressions for functions)

```
1 | (function (x,y) {return x+y})(2,3);
```

匿名函数的作用: (网络资料)

1. 通过匿名函数可以实现闭包
2. 模拟块级作用域, 减少全局变量。执行完匿名函数, 存储在内存中相对应的变量会被销毁, 从而节省内存。再者, 在大型多人开发的项目中, 使用块级作用域, 会大大降低命名冲突的问题, 从而避免产生灾难性的后果。自此开发者再也不必担心搞乱全局作用域了。

3.7 作用域、作用域链、闭包及其用途

作用域：有权访问的变量的集合

ES6 之前，只有函数作用域和全局作用域

ES6 增加了通过 `let` 和 `const` 声明变量的块级作用域，这个区块对这些变量从一开始就形成了封闭作用域，直到声明语句完成，这些变量才能被访问（获取或设置），否则会报错 `ReferenceError`

通过 `let` 声明的变量没有变量提升、拥有暂时性死区，作用于块级作用域

作用域链

对性能有影响，有效的解析越靠前，处理速度越快（不用太多遍历）

参考：<https://www.cnblogs.com/coco1s/p/4017544.html>

（18 年）**闭包：**闭包是函数和执行它的作用域组成的综合体，可以访问它被创建时的上下文环境，即闭包就是能够读取其他函数内部变量的函数

（18 年）**用途：**

- 实现私有成员
- 保护命名空间
- 避免污染全局变量
- 变量需要长期驻留在内存（不销毁的栈环境）

（18 年）**缺点：**

- 比普通函数占用更多的内存，引发内存泄漏；
- 闭包会在父函数外部，改变父函数内部变量的值

解决：闭包不再使用时，要及时释放。

示例：

```
1  var name = "The window";
2
3  var object = {
4    name : "My Object",
5
6    getNameFunc : function(){
7      return function(){
8        return this.name;
9      };
10   }
11 };
12
13 alert(object.getNameFunc()());
```

输出：The Window

```
1  var name = "The window";
2
3  var object = {
4    name : "My Object",
5
```

```
6     getNameFunc : function(){
7         var that = this;
8         return function(){
9             return that.name;
10        };
11    }
12 };
13
14 alert(object.getNameFunc());
```

输出: My Object

闭包参考: http://www.ruanyifeng.com/blog/2009/08/learning_javascript_closures.html

4. DOM

当一个页面被加载之后, 浏览器就生成了一个关于这个页面的 DOM

DOM 是 W3C 的标准, 定义了访问文档的标准, 分为三个不同的部分:

- 核心 DOM - 针对任何结构化文档的标准模型
- XML DOM - 针对 XML 文档的标准模型
- HTML DOM - 针对 HTML 文档的标准模型

The HTML DOM:

是一个 HTML 的标准对象模型和编程接口, 它定义了:

- 将所有 HTML 元素作为对象
- 所有 HTML 元素的属性
- 访问所有 HTML 元素的方法
- 针对所有 HTML 元素的事件

换句话说, HTML DOM 是如何 get, change, add, or delete HTML 元素的标准。

4.1 XHTML/XML 与 DOM 树

页面的元素嵌套在一个树状的对象结构中, 这就是 DOM 树

遍历 DOM 树:

每个节点的 DOM 对象都有如下属性:

- firstChild, lastChild
- childNodes
- nextSibling, previousSibling
- parentNode

4.2 DOM0, DOM2 事件流 (捕获、目标和冒泡)

W3C 的 DOM 事件触发分为三个阶段: (网络资料)

- 事件捕获阶段, 即由最顶层元素 (一般是从 window 元素开始, 有的浏览器是从 document 开始) 开始, 逐次进入 dom 内部, 最后到达目标元素, 依次执行绑定在其上的事件
- 处于目标阶段, 检测机制到达目标元素, 按事件注册顺序执行绑定在目标元素上的事件。

- 事件冒泡阶段，从目标元素出发，向外层元素冒泡，最后到达顶层（window 或 document），依次执行绑定在其上的事件

参考：<https://www.cnblogs.com/guangyan/p/6686013.html>

Dom0 和 DOM2 比较：

- 都可以绑定多个不同事件顺序执行
- 0 级只能绑定不同事件，相同以最后一个为准，2 级则都可以且顺序执行不覆盖

4.3 观察者模式

1. 一种面向对象的设计模式，被观察者维护一个观察者列表，当状态改变时自动通过调用观察者的回调方法通知观察者
2. 主要用户执行分布式事件处理系统
3. MVC 模式中的重要模式
4. 是发布/接收的子集

优点：

- 主体与观察者的抽象耦合
- 支持广播通信

注意：观察者模式可能会导致内存泄漏，称为 lapsed listener problem

5. Ajax

5.1 RIA

RIA：Rich Internet Application，富网络应用，具有高度互动性、丰富的用户体验以及强大的客户端

特点：

- 有桌面应用的功能与特点（高度交互性 GUI）
- 浏览器中运行或沙盒安全环境中运行
- 不需要软件安装，但运行环境还是需要安装

优点：

- 不需要安装
- 升级容易
- 可以通过互联网/内联网轻松获取
- 丰富的响应式的 UI
- Client/Server 平衡
- 异步通讯
- 网络效率高

缺点：

- 对搜索引擎失去可见性（不容易被搜索引擎找到）
- 存在专利问题
- 完整性（一致性）缺失（RIA 通常不能很好地与 HTML 混合）

- 软件开发复杂
- RIA 架构打破了 Web 页面的模式

5.2 同步、异步通信

同步：用户必须等待新页面加载网页中使用的典型通信方式（单击、等待、刷新），几乎所有带有新数据的更改都会导致页面刷新

异步：用户可以在加载数据时保持与页面的交互

Ajax 使通信模式成为可能，使用新数据进行更改，但不刷新页面

5.3 Ajax 请求

`$.ajax()` 返回其创建的 XMLHttpRequest 对象。

5.4 Ajax 优缺点

优点：

1. 更好的交互性和响应性
2. 页面使用起来更加方便
3. 部分修改，减少与服务器的连接次数
4. 节省带宽，因为只需要加载用于更新页面的数据，而不是刷新整个页面
5. 减少网络拥堵

缺点：

1. 标签化页面的用处不大
2. 需要 JS 支持（可能会被用户禁用）
3. 网络延迟减少可用性
4. 无法被搜索引擎发现，SEO 不友好
5. 前进和返回按钮失效
6. 有其安全性限制，只能运行于存储在 web server 上的网页

风险：

- Bookmarking 问题
- 前进和返回按钮的问题
- 安全风险
- 搜索引擎

5.5 安全相关，SOP，跨域

跨域今年很有可能考！！

（18 年）**跨域**：当协议、子域名、主域名、端口号中任意一个不相同时，都算作不同域。

同源 SOP（Same Origin Policy），就是要求域名、协议、端口相同，非同源的脚本不能访问或者操作其他域的页面对象（如 DOM 等）

（18 年）**处理方法**：

- JSONP
 - 利用 `<script>` 元素
- `document.domain` + `iframe` 跨域
 - 仅限主域相同，子域不同的跨域应用场景
 - 实现原理：两个页面都通过 js 强制设置 `document.domain` 为基础主域，就实现了同域。
- `location.hash` + `iframe` 跨域
 - 实现原理：a 欲与 b 跨域相互通信，通过中间页 c 来实现。三个页面，不同域之间利用 `iframe` 的 `location.hash` 传值，相同域之间直接 js 访问来通信。
- `window.name` + `iframe` 跨域
 - 独特之处：`name` 值在不同的页面（甚至不同域名）加载后依旧存在，并且可以支持非常长的 `name` 值（2MB）
- CORS：跨域资源共享
- WebSocket protocol 是 HTML5 一种新的协议。它实现了浏览器与服务器全双工通信，同时允许跨域通讯，是 server push 技术的一种很好的实现。
- `postMessage` 是 HTML5 XMLHttpRequest Level 2 中的 API，且是为数不多可以跨域操作的 `window` 属性之一

5.6 数据格式

XML：

- 极高的互操作性
- 严格格式化
- 容易验证
- 冗长
- 语法含糊

高性能的 Ajax 不会用到 XML

JSON：

- 轻量级且易于解析的数据格式
- 使用 JavaScript 对象和数组文字语法编写。

JSON-P：

- JSON with padding（带有填充的 JSON）
- 使用动态脚本标签插入时，JSON 数据被视为另一个 JS 文件，并作为本机代码执行。为了做到这一点，必须在回调函数中对数据进行包装。

避免使用与性能无关的 JSON-P 的原因：由于 JSON-P 必须是可执行的 JavaScript，因此任何人都可以调用它，并使用动态脚本标记插入将其包含在任何网站中

HTML：

- 作为一种数据格式，速度慢且膨胀

三、服务器端

1. Node.js *

1.1 特点，应用场景

(18 年) **why nodejs?**

- 非阻塞 I/O
- V8 Javascript 引擎
- 具有事件循环的单线程
- 超过一百万个模块
- Windows, Linux, Mac
- 1 种用于前端和后端的语言
- 活跃社区

(18 年) **缺点:**

- 不适合 CPU 密集型应用: 由于 JavaScript 单线程的原因, 如果有长时间运行的计算 (比如大循环), 将会导致 CPU 时间片不能释放, 使得后续 I/O 无法发起;
- 只支持单核 CPU, 不能充分利用 CPU
- 可靠性低, 一旦代码某个环节崩溃, 整个系统都崩溃

(18 年) **应用场景:**

- 网站 (如 express/koa 等)
- im 即时聊天 (socket.io)
- api (移动端, pc, h5)
- HTTP Proxy (淘宝、Qunar、腾讯、百度都有)
- 前端构建工具 (grunt/gulp/bower/webpack/fis3...)
- 写操作系统 (NodeOS)
- 跨平台打包工具 (PC 端的 electron、nw.js, 比如钉钉 PC 客户端、微信小程序 IDE、微信客户端, 移动的 cordova, 即老的 Phonegap, 还有更加有名的一站式开发框架 ionicframework)
- 命令行工具 (比如 cordova、shell.js)
- 反向代理 (比如 anyproxy, node-http-proxy)

什么时候不用 nodejs:

- 在服务端进行繁重和密集的 CPU 运算时
- 有数据库时, nodejs 的关系数据库工具还处于早期阶段

1.2 基本原理

单线程, 异步 I/O, 事件驱动

Node.js 的单线程并不是真正的单线程, 只是开启了单个线程进行业务处理 (CPU 的运算), 同时开启了其他线程专门处理 I/O。当一个指令到达主线程, 主线程发现有 I/O 之后, 直接把这个事件传给 I/O 线程, 不会等待 I/O 结束后, 再去处理下面的业务, 而是拿到一个状态后立即往下走, 这就是“单线程”、“异步 I/O”

I/O 处理完之后会有一个回调事件，这个事件会放在一个事件处理队列里头，在进程启动时 node 会创建一个类似于 while(true) 的循环，它的每一次轮询都会去查看是否有事件需要处理，是否有事件关联的回调函数需要处理，如果有就处理，然后加入下一个轮询，这就是所谓的“事件驱动”

(网络资料)

事件驱动：为需要处理的事件编写相应的事件处理程序，代码在事件发生时执行

事件驱动模型：

- 事件源：产生事件的地方（html 元素）
- 事件：点击、鼠标操作、键盘操作等等
- 事件对象：当某个事件发生时，可能会产生一个事件对象，该事件对象会封装好该事件的信息，传递给事件处理程序
- 事件处理程序：响应用户事件的代码

五、优化 *

1. 基准测试/性能分析

基准测试（Benchmarking）：通过设计合理的测试方法，选用合适的测试工具和被测系统，实现对某个特定目标场景的某项性能指标进行定量的和可对比的测试

测试工具：

- Apache Bench：apache 自带的一款功能强大的测试工具
- Siege：一款开源的压力测试工具
- http_load
- 阿里云性能测试
- Tsung：一个开源的支持多协议的分布式压力测试工具
- JMeter

影响基准测试数据的因素：

- 地理位置
- 网络问题
- 响应数据大小
- 代码处理
- 浏览器行为
- web 服务器配置

性能监控指标：

- FP（First Paint，首次绘制）：对于应用页面，首次出现视觉上不同于跳转之前内容的时间点，或者说是页面发生第一次绘制的时间点
- FCP（First Contentful Paint，首次内容绘制）：指浏览器完成渲染 DOM 中第一部分内容（可能是文本、图像或其他任何元素）的时间点，此时用户应该在视觉上有直观的感受。
 - 注意：只有首次绘制文本、图片（包含背景图）、非白色的 canvas 或 SVG 时才被算作 FCP
- FMP（First Meaningful Paint，首次有意义绘制）：指页面关键元素的渲染时间
 - 没有标准化定义，因为关键元素可以由开发者自行定义。

- FMP 本质上是通过一个算法来猜测某个时间点可能是 FMP，所以有时候不准
- 首屏时间：进入页面之后，应用渲染完成整个手机屏幕（未滚动之前）内容的时间
- 用户可交互时间：用户可以与应用进行交互的时间
- 总下载时间：页面所有资源加载完成所需要的时间
- 自定义指标：由于应用特点不同，可以根据需求自定义时间

FP 与 FCP 这两个指标之间的主要区别：

- FP 是当浏览器开始绘制内容到屏幕上的时候，只要在视觉上开始发生变化，无论是什么内容触发的视觉变化，在这一刻，这个时间点，叫做 FP。
- 相比之下，FCP 指的是浏览器首次绘制来自 DOM 的内容。例如：文本，图片，SVG，canvas 元素等，这个时间点叫 FCP
- FP 和 FCP 可能是相同的时间，也可能是先 FP 后 FCP

2.基本原理

两个准则：

- 消除或减少不必要的网络延迟
- 将需要传输的数据压缩至最少

(18 年) 为什么要对 web 进行优化：

- 页面加载如果超过 3 秒，40% 的人会退出，80% 的人不会再重新打开，几乎一半的人会告诉其他人自己访问该页面时不愉快的体验
- 网站的速度对 SEO 的效果十分重要，因为更快的网站更容易被爬取和访问，并且更有可能在搜索结果的高位
- 网站越快，用户的黏性、忠诚度、转化率就越高

Web 性能要点：

- 延迟和带宽对 Web 性能的影响
- 传输协议 (TCP) 对 HTTP 的限制
- HTTP 协议自身的功能和缺陷
- Web 应用的发展趋势和性能需求
- 浏览器局限性和优化思路

3. 优化思路，技术，方法

优化思路：

1. 浏览器渲染流畅
2. 减少 http 请求
3. 优化单个 Http 请求
 - 有效利用缓存
 - 收发数据大小
4. 缩短请求时间

核心优化策略：

- 基于文档的优化：熟悉网络协议，了解文档、CSS 和 JavaScript 解析管道，发现和优先安排关键网络资源，尽早分派请求并取得页面，使其尽快达到可交互的状态。主要方法是优先获取资源、提前解析等
- 推测性优化：浏览器可以学习用户的导航模式，执行推测性优化，尝试预测用户的下一次操作。然后，预先解析 DNS、预先连接可能的目标

浏览器优化利用的四种技术：（要么是预先要么是猜测）

- 资源预取和排定优先次序：文档、CSS 和 JavaScript 解析器可以与网络协议层沟通，声明每种资源的优先级——初始渲染必需的阻塞资源具有最高优先级，而低优先级的请求可能会被临时保存在队列中。
- DNS 预解析：对可能的域名进行提前解析，避免将来 HTTP 请求时的 DNS 延迟。预解析可以通过学习导航历史、用户的鼠标悬停，或其他页面信号来触发。
- TCP 预连接：DNS 解析之后，浏览器可以根据预测的 HTTP 请求，推测性地打开 TCP 连接。如果猜对的话，则可以节省一次完整的往返（TCP 握手）时间。
- 页面预渲染：某些浏览器可以让我们提示下一个可能的目标，从而在隐藏的标签页中预先渲染整个页面。这样，当用户真的触发导航时，就能立即切换过来

方法：

- 减少 DNS 查询
- 重用 TCP 连接
- 减少重定向次数
- 使用 CDN，将静态资源部署在地理位置近的地方
- 消除不必要的资源
- 合理使用缓存机制
- 压缩、减少数据量
- 减少不必要的请求字节
- 并行请求和响应处理
- **使用针对特定协议的优化**（18 年）
 - 对于 HTTP 1.x：
 - 利用 HTTP 管道
 - 采用域名分区
 - 将资源分散到多个来源
 - 打包资源以减少 HTTP 请求
 - 拼接和精灵图
 - 嵌入小资源
 - 直接在父文档中嵌入小资源，从而减少请求数量
 - 对于 HTTP 2.0：
 - HTTP 2.0 的主要重点是提升传输性能，并在客户端和服务端之间实现更低的延迟和更高的吞吐量
 - 少发数据、削减请求、根据无线网络情况调整资源供给
 - 杜绝和忘记域名分区、文件拼接、图片精灵等不良的习惯

精灵图：将网页中需要的零星的小图片集成到一个大的图片中。可以方便对小图标的统一管理，减少对浏览器的请求次数，避免网页的延迟

课件上的其他问题

1. Web Server 基本原理

本质是：接收数据 -> Http 解析 -> 逻辑处理 -> Http 封包 -> 发送数据

基本流程如下：

1. 与浏览器建立 TCP 连接
2. 浏览器将用户事件按照 Http 协议格式打包为数据包，确认对端可写后推入 Internet，包经过网络传至 Server
3. Server 根据 Http 协议格式解析数据包
4. 解析得到用户事件后处理事件逻辑
5. 按照 Http 协议格式将结果数据打包，确认对端可写后推入 Internet，包经过网络传至浏览器
6. 浏览器根据 Http 协议格式解析数据包，根据数据格式展示相应结果

2. 为什么要使用 XHTML 和 Web 标准？

- 更加严格和结构化的语言
- 在不同的 Web 浏览器之间更具互相操作性
- 页面以后展示正确的可能性更高
- 可以与其他 XML 数据交换 SVG（图形）、MathML、MusicML 等

3. Canvas 和 SVG 的比较

Canvas	SVG
Resolution dependent	Resolution independent
No support for event handlers	Support for event handlers
文本渲染能力差	最适合具有大渲染区域的应用程序（谷歌地图）
结果可以保存为 png 或 jpg	如果复杂，则渲染速度慢（任何经常使用 DOM 的东西都会很慢）
非常适合图形密集型游戏	不适合游戏应用程序

4. BFC 的布局规则

BFC：Block formatting context，直译为“块级格式化上下文”

- 内部的 Box 会在垂直方向，一个接一个地放置。
- Box 垂直方向的距离由 margin 决定。属于同一个 BFC 的两个相邻 Box 的 margin 会发生重叠。
- 每个盒子（块盒与行盒）的 margin box 的左边，与包含块 border box 的左边相接触（对于从左往右的格式化，否则相反），即使存在浮动也是如此。

- BFC 的区域不会与 float box 重叠。
- BFC 就是页面上的一个隔离的独立容器，容器里面的子元素不会影响到外面的元素。反之也如此。
- 计算 BFC 的高度时，浮动元素也参与计算。

5. IFC 的影响

IFC 对布局产生的影响主要有以下三个方面：

- 一个 IFC 内的元素都是水平排列的。
- 横向的 margin、border、padding 属性对于这些元素都是有效的。
- 垂直方向可以调整对齐方式。

6. 最佳可访问性和可读性

编码时保持良好的 HTML 源码顺序非常重要，原因：

- 网站技术故障时不能正确显示 CSS 样式，或移动和无线网络环境下，带宽有限，导致浏览器中出现没有样式的 HTML
- 资源顺序对网站的可访问性起着重要作用，因为对于盲人用户，当代码有一定的逻辑顺序时，能够快速跳过页眉和导航区域，直达页面的主要内容。
- 确保网站的主导航链接和主要网页内容在资源排序中排在最前面，这样可以帮助搜索引擎优化。

7. 前端的趋势（18 年）

（网络资料）2020 年的趋势

1. TypeScript 爆发增长
2. 三大框架 React 当先（React、Angular、Vue.js）
3. WASM（WebAssembly）崭露头角
4. Low-Code
5. 全栈开发
6. DevOps 渐进增强
7. WebRTC 持续升温

8. 使用 em 排版尺寸的重要性

- 当设备屏幕上的排版太大或太小时，用户都可以很容易地进行调整，这对于有视力障碍的用户来说是一个重要的可访问特性。
- 在 CSS 控制下，相对类型的大小可以很容易地缩放，在 <html> 或 <body> 标签中使用 font-size 元素，就可以很容易地对整个页面排版尺寸进行缩放。通过调整 em 的值，可以立刻放大或缩小页面上的所有排版，这在响应式设计中是一种非常有用的功能，可以快速缩放页面排版，以轻松匹配整个屏幕尺寸和分辨率。

9. 面包屑导航

适用条件：

- 层级较深的网站，如果只有一级分类的话，通过主导航就可以起到快速定位的作用。比如“豆瓣网”类型扁平构架的网站就没有使用面包屑导航。

- 独立不交叉的网站结构，由于面包屑网站导航路径是线性结构的，因此网站内容必须划分的非常清晰，且不存在交叉；否则，面包屑导航的路径就不是唯一的，同一分类可能出现在不同的路径中，让用户感到困惑

作用：

1. 让用户了解当前所处位置，以及当前页面在整个网站中的位置。
2. 体现了网站的架构层级，能够帮助用户快速学习和了解网站内容和组织方式，从而形成很好的位置感。
3. 提供返回各个层级的快速入口，方便用户操作。
4. Google 已经将面包屑导航整合到搜索结果里面，因此优化面包屑导航每个层级的名称，多使用关键字，都可以实现 SEO 优化。面包屑路径对于提高用户体验来说，是很有帮助的。
5. 方便用户，面包屑主要用于为用户提供导航一个网站的次要方法，通过为一个大型多级网站的所有页面提供面包屑路径，用户可以更容易的定位到上一次目录，引导用户通行；
6. 减少返回到上一级页面的点击或操作，不用使用浏览器的“返回”按钮或网站的主要导航来返回到上一级页面；
7. 不用常常占用屏幕空间，因为它们通常是水平排列以及简单的样式，面包屑路径不会占用页面太多的空间。这样的好处是，从内容过载方面来说，它们几乎没有任何负面影响；
8. 降低跳出率，面包屑路径会是一个诱惑首次访问者在进入一个页面后去浏览这个网站的非常好的方法。比如说，一个用户通过谷歌搜索到一个页面，然后看到一个面包屑路径，这将会诱使用户点击上一级页面去浏览感兴趣的相关主题。这样可以降低网站的总体跳出率。
9. 有利于百度蜘蛛（百度搜索引擎的一个自动程序）对网站的抓取，蜘蛛直接沿着那个链走就可以了，很方便。
10. 面包屑有利于网站内链的建设，用面包屑大大增加了网站的内部连接，提高用户体验。

10. Vanilla JS (18 年)

概念：Vanilla JS 是为搭建功能强大的 JS 应用的一个快速的、轻量的、跨平台的框架

JS 框架的优点：

- 封装了复杂困难的代码
- 能加快开发速度，更快完成项目
- 让人更专注于产品内容的价值，而不是实现过程
- 让合作更简单，大家都对基础代码有共同的理解
- 强迫你练习，多实践，顺能生巧

JS 框架的问题：

- 每个项目的开发都会遇到框架文档没有说明的问题，这时就要深入框架内部查找原因，就需要对原生 JS 的掌握
- 新框架频繁发布，更新快速，一旦确定了项目的技术栈，随着时间，如何更新升级是个问题

why Vanilla JS：

- 学会 Vanilla JS 能真正理解 JS 框架，甚至能为其贡献代码，还能帮助选择合适的框架
- 如果不知道 Web 基本原则，如何应对语言本身的演变和新框架的不断到来是个问题
- 知道纯 JS 将成为一个能够（不用疯狂搜索原因）解决复杂问题的关键工程师
- 增加通用能力和生产力，不管在前端还是后端

- 创新的工具，而不只是执行
- 指导什么时候使用或者不使用框架
- 更好地了解浏览器和计算机工作原理

11. 客户端编程和服务端编程的比较

客户端编程：

- 可用性：可以修改页面而不必发回服务器
- 效率：可以在不等待服务器的情况下对页面进行小的、快速的更改
- 事件驱动：可以响应用户操作，如点击和按键
- 平台独立：代码可以被任何启用脚本的浏览器解释

服务端编程：

- 安全：可以访问客户端看不到的服务器的私有数据
- 兼容性：不受浏览器兼容性问题的影响
- 强大：可以编写文件，开启与服务器的连接，连接数据库等

12. JS 的优点

- 性能：
 - JIT：just in time，即时编译，当某段代码即将第一次被执行时进行编译
 - 垃圾收集和动态绑定
- 对象：
 - 使用原型继承模型
- 语法：
 - 和 C 族语言非常相似，比如 C++、Java、C#、PHP
- "first-class" functions：
 - JS 中几乎所有的东西都是对象，包括函数
- 事件：
 - 在浏览器中，所有内容都在事件循环中运行
- 可复用性：
 - 最可移植、可重用的代码
 - 可以模块化和封装

13. 什么是 JS 的 cookies

- cookies 让你在 web 页面上存储用户数据
- cookies 就是数据，在你的电脑上以小文本文件的形式存储
- cookies 的发明是为了解决如何记住用户信息的问题
- cookies 是以 name-value 对保存的
- 当浏览器从服务器请求网页时，属于该网页的 cookies 会添加到请求中，这样服务器就可以获得必要的信息来“记住”有关用户的信息

14. JS 的作用域

局部变量：

- 局部变量在函数内部声明，便仅在函数内部
- 作用域仅在函数中
- 当一个函数开始的时候，局部变量产生，函数执行结束就消亡

全局变量：

- 在函数外部声明的变量为全局变量
- 全局变量在任何地方都可以访问到
- **自动全局化**：当没有声明就直接赋值的变量，会自动转化成一个全局变量

变量的作用域：

- 局部变量从函数的开始到结束
- 全局变量从声明到关闭页面

15. 同步式 IO 和异步式 IO 的特点

同步式 I/O（阻塞式）	异步式 I/O（非阻塞式）
利用多线程提供吞吐量	单线程即可实现高吞吐量
通过时间片分割和线程调度利用多核 CPU	通过功能划分利用多核 CPU
需要由操作系统调度多线程使用多核 CPU	可以将单进程绑定到单核 CPU
难以充分利用 CPU 资源	可以充分利用 CPU 资源
内存轨迹大，数据局部性弱	内存轨迹小，数据局部性强
符合线性的编程思维	不符合传统编程思维

异步式也就是 nodejs

16. cookie 和会话

cookie 的想法很简单：服务器发送一点信息，浏览器在一段可配置的时期内保存它。

关于 cookie：

- cookie 对用户来说不是加密的
 - 服务器向客户端发送的所有 cookie 都能被客户端查看
- 用户可以删除或禁用 cookie
- 一般的 cookie 可以被篡改
 - 要确保 cookie 不被篡改，请使用签名 cookie
- cookie 可以用于攻击
 - XSS
- 如果滥用 cookie，用户会注意到
- 如果可以选择，会话要优于 cookie

- 大多数情况下，可以用会话维持状态，一般来说这样做是明智的。并且会话更容易，你不用担心会滥用用户的存储，而且也更安全。当然，会话要依赖 cookie，但如果你使用会话，Express 会帮你做很多工作。

会话的用途：

- 跨页保存用户的偏好
- 提供用户验证信息

实现会话的方法：

- 把所有东西都存在 cookie 里
- 只在 cookie 里存一个唯一标识，其他东西都存在服务器上。

17. CSS 媒体查询

有条件地检测用户显示屏的各个方面，然后根据这些条件有选择地加载出样式表，并提供最合适的布局、排版和图形。

媒体查询可用于检测很多事情：

- viewport（视窗）的宽度与高度
- 设备的宽度与高度
- 朝向（智能手机横屏，竖屏）
- DPI 分辨率

18. 移动优先

优先内容和功能，即优先显示最重要的内容和功能，如果空间允许，再逐步加入次要内容和功能。

好处：

- 通用访问
 - 只要移动端做的好，即使用户使用的是旧版本浏览器、没有 Javascript 或者关闭了 Javascript 的浏览器，或为视力残障人士设计的读屏浏览器，也能看到一个拥有基本功能的网站。
 - 移动优先是渐进增强理念的良好范例，所有用户都能访问核心内容和功能。不存在不能访问的情况

19. 同步和异步比较

同步	异步
等待一个操作执行完之后，再执行下一个	从不等待每个操作完成，一次执行所有操作，有结果就处理
一步一步地执行	用回调处理结果

20. http

HTTP 是支持 Web 浏览器和 Web 服务器之间通信的协议。

工作模式：请求（request）和响应（response）

请求方法：GET、POST

HTTP 是无状态的，服务器不会对之前客户端的请求进行保存，即工作方式简单

HTTP 和 HTTPS 的区别：

- HTTPS URL 以 “https://” 开头，默认使用端口 443；HTTP URL 以 “http://” 开头，默认使用端口 80
- HTTP 没有加密，容易受到中间人攻击和窃听攻击，攻击者可以访问网站帐户和敏感信息，并修改网页以注入恶意软件或广告；HTTPS 旨在抵御此类攻击，并被认为是安全的（旧的、不推荐使用的 SSL 版本除外）

SPDY 的目标：

- 实现 PLT（page load time）减少 50%
- 避免网站作者对内容进行任何更改
- 最大限度地减少部署的复杂性，避免改变现有的网络设施
- 与开源社区合作开发这个新协议
- 收集真实的性能数据来验证实验协议

SPDY 和 HTTP 的联系：

- SPDY 不是来取代 HTTP 的
- SPDY 是 HTTP 和 HTTPS 协议的一条有效通道

请求头：

- 提供给服务器关于客户端的信息
- 包括：客户端种类、将被接受的内容的种类、请求发出者
- （18 年）常用请求头：Accept、Accept-Charset、Accept-Encoding、Accept-Language、TE、Expect、If-Match

响应头：

- 常用响应头：Accept-Patch、Age、Allow、Connection、Link

（18 年）状态码：

- 1xx：信息性
- 2xx：成功
 - 200：成功
- 3xx：重定向
 - 301：是一种永久性的重定向，搜索引擎会删除原页面，收录重定向的页面，并转移权重
 - 302：是一种临时性的重定向，大部分搜索引擎把它作为内部的重定向，不会缓存重定向的结果。一些旧客户端会错误地将请求方法转换为 GET
 - 307：临时重定向，确保请求方法和消息主体不会发生变化
- 4xx：客户端错误
 - 400：Bad Request
 - 401：Unauthorized
 - 403：forbidden
 - 404：Not Found，页面不存在或链接错误，搜索引擎放弃对该链接的索引
- 5xx：服务器错误

- 500: Internal Sever Error

什么情况下使用 301 重定向?

- 网站更换域名时, 通过 301 永久重定向将旧域名重定向至新域名, 挽回流量损失和 SEO
- 当出于需要删除网站中的某些目录时, 比如我要删除我博客下的博客导航, 这时就可以用 301 永久重定向到网站首页
- 如果你有多个闲置域名需要指向同一网站时, 通过 301 永久重定向可以实现
- 你打算实现网址规范化

302 劫持: 302 重定向的危害之一。一个不道德的人在他自己的网址 A 做一个 302 重定向到你的网址 B, 出于某种原因, 搜索引擎搜索结果所显示的仍然是网址 A, 但是所用的网页内容却是你的网址 B 上的内容, 这种情况就叫做网址 URL 劫持

流: 建立的连接内的字节的双向流。

优先级: 31bit 的值 (0 最高)

信息: 映射到逻辑消息的帧的完整序列。

帧: HTTP 2.0 中的最小通信单元, 每个通信单元都包含一个帧头, 该帧头至少标识了帧所属的流。

流控制 (flow control) :

- 流控制是逐跳 (hop-by-hop) 的, 而不是端到端 (end-to-end) 的
- 流控制基于 window update frames
- 流控制窗口大小由 WINDOW_UPDATE frame 更新
- 流控制是定向的
- 接收方可以禁用流控制, 无论是对单个流还是对整个连接

server push 的好处:

- 推送资源可以被客户端缓存
- 推送资源可以被客户端拒绝
- 推送资源可以被不同页面重用
- 推送资源可以被服务器按优先级排序

21. Express

Express 是最流行的 node web 框架, 它是许多其他流行的节点 web 框架的底层库。它提供了机制:

1. 在不同的 URL 路径 (路由) 中使用不同 HTTP 动词的请求编写处理程序。
2. 与“视图”呈现引擎集成, 以便通过将数据插入模板来生成响应。
3. 设置常见的 web 应用程序设置, 比如用于连接的端口, 以及用于呈现响应的模板的位置。
4. 在请求处理管道的任何位置添加额外的请求处理“中间件”

特点:

- 精简
- 灵活
- web 程序框架
- 单页 web 程序

- 多页和混合的 web 程序

22. Babel

Babel 是一个 JavaScript 编译器

Babel 是一个工具链，主要用于将 ECMAScript 2015+ 版本的代码转换为向后兼容的 JavaScript 语法，以便能够运行在当前和旧版本的浏览器或其他环境中

功能：

- 语法转换
- 通过 Polyfill 方式在目标环境中添加缺失的特性
- 源码转换

23. 插件的坏处（14 年）

1. 使用需要下载
2. 相对于 built-in 而言，bolt-on 更加好
3. 是潜在的 attack vector，存在代理中毒的问题
4. 如乔布斯所言：Mac 崩溃的常见原因就是 Flash 插件

24. 平滑降级

当一些老式浏览器不支持新标签时会把标签以自己能支持的相对低级的标签形式显示，如 img 中的 alt 属性，input 当做 type=text

名词解释（缩写）

WWW

World Wide Web，万维网。存储在网络计算机中，数据量巨大的文档的集合，也是一组软件和协议的集合，WWW 服务器通过 HTML 把信息组织成超文本，利用链接从一个站点跳到另一个站点

IP

Internet Protocol，网际互连协议，是为计算机网络相互连接进行通信而设计的协议，可以用来唯一标识互联网中的节点

TCP

Transmission Control Protocol，传输控制协议，是一种面向连接的、可靠的、基于字节流的传输层通信协议

UDP

User Datagram Protocol，用户数据报协议，为应用程序提供了一种无需建立连接就可以发送封装的 IP 数据报的方法

DNS (2018)

Domain Name System，域名系统，分为三级域名、二级域名、顶级域名。是因特网上作为域名和 IP 地址相互映射的一个分布式数据库，通过 DNS 服务器将域名转化为 IP 地址，然后访问

RR

resource records，资源记录，分布式数据库的数据库记录

URI

Uniform Resource Identifier，统一资源标识符，是一个用于标识某一互联网资源名称的字符串。这种标识允许用户对任何（包括本地和互联网）的资源通过特定的协议进行交互操作，由包括确定语法和相关协议的方案所定义

URL (2018)

Uniform Resource Locator，统一资源定位符，是对可以从互联网上得到的资源的位置和访问方法的一种简洁的表示，是互联网上标准资源的地址。互联网上的每个文件都有一个唯一的 URL，它包含的信息指出文件的位置以及浏览器应该怎么处理它

URN

Uniform Resource Name，统一资源名称，唯一标识一个实体的标识符，但不能给出实体的位置。系统可以先在本地寻找一个实体，在它试着在 Web 上找到该实体之前。它也允许 Web 位置改变，然而这个实体却还是能够被找到

QUIC

Quick UDP Internet Connection，是谷歌制定的一种基于 UDP 的低时延的互联网传输层协议

HTML

HyperText Markup Language，超文本标记语言，是一种制作 web 页面的标准语言

HTML5 (2018)

- HTML5 是下一代 HTML 标准
- HTML5 仍处于完善之中。然而，大部分现代浏览器已经具备了 HTML5 支持
- HTML5 是 W3C 与 WHATWG 合作的结果

XML (2014)

extensible markup language，可扩展标记语言，标准通用标记语言的子集，是一种用于标记电子文件使其具有结构性的标记语言

XHTML

Extensible Hypertext Markup Language，可扩展超文本标记语言，是一种基于 XML 的标记语言，表现方式和 HTML 类似，但语法更加严格

SEO

search engine optimisation，搜索引擎优化，指通过对网站内部调整优化及站外优化，使网站满足搜索引擎收录排名需求，在搜索引擎中关键词排名提高，从而把精准用户带到网站，获得免费流量，产生直接销售或品牌推广

SVG

scalable vector graphics，可缩放矢量图形，基于可扩展标记语言（标准通用标记语言的子集），用于描述二维矢量图形的一种图形格式

CSS

cascading style sheets，层叠样式表，是一种用来表现 HTML 或 XML 等文件样式的计算机语言

BFC

Block formatting context，块级格式化上下文。它是一个独立的渲染区域，只有 Block-level box 参与，它规定了内部的 Block-level Box 如何布局，并且与这个区域外部毫不相干

DOM

Document Object Model，文档对象模型，是 W3C 的标准，定义了访问 HTML 和 XML 文档的标准，将 HTML 文档表达为树结构，HTML DOM 定义了访问和操作 HTML 文档的标准方法。W3C DOM 标准被分为 3 个不同的部分：

- 核心 DOM - 针对任何结构化文档的标准模型
- XML DOM - 针对 XML 文档的标准模型
- HTML DOM - 针对 HTML 文档的标准模型

BOM

Browser Object Model，浏览器对象模型，提供了独立于内容的、可以与浏览器窗口进行互动的对象结构。BOM 由多个对象组成，其中代表浏览器窗口的 window 对象是 BOM 的顶层对象，其他对象都是该对象的子对象。在浏览器中的一些操作都可以使用 BOM 的方式进行编程处理，比如：刷新浏览器、后退、前进、在浏览器中输入 URL 等

TDZ

temporal dead zone，暂时性死区，即代码块开始到变量声明语句完成之间的区域

JIT

just in time，即时编译，当某段代码即将第一次被执行时进行编译

MVC

Model-View-Controller，模型-视图-控制器，是一种软件设计典范，用一种业务逻辑、数据、界面显示分离的方法组织代码，将业务逻辑聚集到一个部件里面，在改进和个性化定制界面及用户交互的同时，不需要重新编写业务逻辑。MVC 被独特地发展起来用于映射传统的输入、处理和输出功能在一个逻辑的图形化用户界面的结构中

HTTP (2014)

hypertext transfer protocol, 超文本传输协议。

HTTP 是一个应用层协议, 具有分布式、超媒体信息系统所需的轻便和高速的特点

HTTPS

HTTPS (也称 HTTP over TLS、HTTP over SSL、或 HTTP Secure), 是用于在互联网上广泛使用的计算机网络安全通信的协议, 在HTTP的基础上通过传输加密和身份认证保证了传输过程的安全性

SPDY

是谷歌开发的基于 TCP 的会话层协议, 以最小化网络延迟、提升网络速度来优化用户的网络使用体验

AJAX (2014)

Asynchronous JavaScript and XML (异步的 JavaScript 和 XML), 是在不重新加载整个页面的情况下, 与服务器交换数据并更新部分网页的技术。

RIA (2014)

Rich Internet Application, 富网络应用, 具有高度互动性、丰富的用户体验以及强大的客户端

SOP

Same Origin Policy, 同源策略, 要求域名、协议、端口相同, 非同源的脚本不能访问或者操作其他域的页面对象 (如 DOM 等)

WebSocket

WebSocket protocol 是 HTML5 一种新的协议。它实现了浏览器与服务器全双工通信, 同时允许跨域通讯, 是 server push 技术的一种很好的实现

WPO

web performance optimization, web 性能优化, 即增加网页的加载速度, 一般有减少 HTTP 请求, 使用 CDN 加速, 利用缓存等方法

CDN

Content Delivery Network, 内容分发网络, 基本思路是尽可能避开互联网上有可能影响数据传输速度和稳定性的瓶颈和环节, 使内容传输更快、更稳定

XSS (2014)

cross site scripting, 跨站脚本攻击。恶意攻击者往 web 页面里插入恶意的 html 代码, 当用户浏览该页时, 嵌入其中的 html 代码就会被执行, 从而达到恶意攻击用户的特殊目的

RPC

remote procedure call protocol，远程过程调用协议。这是一种通过网络从远程计算机程序上请求服务，而不需要了解底层网络技术的协议。RPC 协议假定某些传输协议的存在，如 TCP 或 UDP，为通信程序之间携带信息数据。在 OSI 网络通信模型中，RPC 跨越了传输层和应用层，使得开发包括网络分布式多程序在内的应用程序更加容易

RTT

round-trip time，往返时延，表示从发送端发送数据开始，到发送端收到来自接受端的确认，总共经历的时间

RWD

Responsive Web Design，响应式 Web 设计，是一种新的网站设计模式，以此构建的网站可以自动适应不同的访问设备，方便用户阅读和导航浏览，减少用户的放大、缩小、滑动操作，从而提供完整而友好的用户体验

课件上没有的问题

1. 在浏览器里输入 URL 并回车后会发生什么

- URL 解析
 - 地址解析
 - 其他操作（安全检查）
- DNS 查询
- TCP 连接
 - 建立连接，三次握手
- 处理请求
- 接受响应
- 渲染页面